

3

Desenvolvimento ágil de software

OBJETIVOS

O objetivo deste capítulo é apresentá-lo aos métodos de desenvolvimento ágil de software. Ao ler este capítulo, você:

- ▶ compreenderá a lógica dos métodos de desenvolvimento ágil de software, o manifesto ágil e as diferenças entre o desenvolvimento ágil e o dirigido por plano;
- ▶ conhecerá práticas importantes do desenvolvimento ágil, como as histórias do usuário, a refatoração, a programação em pares e o desenvolvimento com testes *a priori* (*test-first*);
- ▶ compreenderá desde a abordagem Scrum até o gerenciamento ágil de projetos;
- ▶ compreenderá as questões de escalabilidade dos métodos de desenvolvimento ágil e a combinação das abordagens ágeis com as dirigidas por plano no desenvolvimento de grandes sistemas de software.

CONTEÚDO

- 3.1 Métodos ágeis
- 3.2 Técnicas de desenvolvimento ágil
- 3.3 Gerenciamento ágil de projetos
- 3.4 Escalabilidade dos métodos ágeis

As empresas de hoje em dia operam em um ambiente global em rápida mudança. Elas precisam responder às novas oportunidades e mercados, às mudanças nas condições econômicas e ao surgimento de produtos e serviços concorrentes. O software faz parte de quase todas as operações de negócios, então novo software tem de ser desenvolvido rapidamente, para que seja possível tirar vantagem das novas oportunidades e responder à pressão da concorrência. A entrega e desenvolvimento rápidos são, portanto, os requisitos mais importantes da maioria dos sistemas de negócios. Na verdade, as empresas podem estar dispostas a negociar a qualidade e comprometer requisitos se puderem implantar um novo software rapidamente.

Como essas empresas operam em um ambiente dinâmico, é praticamente impossível derivar um conjunto completo de requisitos de software estáveis. Esses requisitos mudam porque os clientes acham impossível prever como um sistema

irá afetar as práticas profissionais, como irá interagir com outros sistemas e quais operações de usuário devem ser automatizadas. Pode ser que só depois que um sistema tenha sido entregue, e os usuários tenham adquirido experiência com ele, que os verdadeiros requisitos fiquem claros. Mesmo assim, fatores externos orientam sua mudança.

Os processos de desenvolvimento de software dirigidos por plano que especificam completamente os requisitos e depois projetam, constroem e testam um sistema não são voltados para o desenvolvimento rápido de software. À medida que os requisitos mudam ou que problemas de requisitos são descobertos, o projeto ou a implementação do sistema precisam ser retrabalhados e testados novamente. Como consequência, um processo convencional em cascata ou baseado em especificação normalmente é demorado e o software final é entregue ao cliente muito depois do prazo originalmente estipulado.

Para alguns tipos de software — como os sistemas de controle críticos em segurança, para os quais uma análise completa do sistema é essencial —, essa abordagem dirigida por plano é a mais indicada. No entanto, em um ambiente empresarial dinâmico, isso pode ser problemático. Quando o software finalmente estiver disponível para uso, o motivo original de sua aquisição pode ter mudado tão radicalmente que ele acaba sendo inútil. Portanto, especialmente no caso de sistemas de negócio, os processos de desenvolvimento e entrega rápidos são essenciais.

A necessidade de desenvolvimento rápido de software e de processos que possam lidar com requisitos que mudam foi reconhecida há muitos anos (LARMAN; BASILI, 2003). Entretanto, o desenvolvimento mais rápido do software só decolou no final dos anos 1990, quando surgiu a ideia de “métodos ágeis” como a Programação Extrema — do inglês *Extreme Programming*, ou XP — (BECK, 1999), o Scrum (SCHWABER; BEEDLE, 2001) e o DSDM — do inglês *Dynamic Systems Development Method* — (STAPLETON, 2003).

O desenvolvimento rápido de software passou a ser conhecido como desenvolvimento ágil, ou métodos ágeis. Esses métodos são concebidos para produzir software útil de maneira rápida. Todos eles compartilham uma série de características comuns:

1. Os processos de especificação, projeto e implementação são intercalados. Não há especificação detalhada do sistema e a documentação do projeto é minimizada ou gerada automaticamente pelo ambiente de programação utilizado para implementar o sistema. O documento de requisitos do usuário é uma definição resumida contendo apenas as características mais importantes do sistema.
2. O sistema é desenvolvido em uma série de incrementos. Os usuários finais e outros *stakeholders* estão envolvidos na especificação e avaliação de cada um deles. Além de mudanças no software, eles também podem propor novos requisitos para serem implementados em uma versão posterior do sistema.
3. O amplo apoio de ferramentas é usado para ajudar no processo de desenvolvimento. Podem ser utilizadas ferramentas de teste automatizado, ferramentas de apoio ao gerenciamento de configuração e à integração de sistemas, além de ferramentas para automatizar a produção da interface com o usuário.

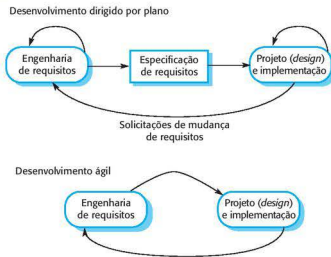
Os métodos ágeis se baseiam no desenvolvimento incremental; os incrementos são pequenos e, normalmente, novas versões do sistema são criadas e disponibilizadas para os clientes a cada duas ou três semanas, para que seja possível obter deles um *feedback* rápido nos requisitos que mudam. Além disso, esses métodos

minimizam a documentação usando comunicação informal em vez de reuniões formais com documentos escritos.

As abordagens ágeis de desenvolvimento de software consideram o projeto (*design*) e a implementação como as atividades centrais no processo de software. Elas incorporam outras tarefas a essas atividades, como a eliciação dos requisitos e os testes. Por outro lado, uma abordagem dirigida por plano identifica etapas diferentes no processo de software, com resultados associados a cada uma delas. Esses dados são utilizados como base para o planejamento da atividade de processo seguinte.

A Figura 3.1 mostra as diferenças fundamentais entre as abordagens ágil e dirigida por plano na especificação de sistemas. Em um processo de desenvolvimento de software dirigido por plano, a iteração ocorre dentro das atividades, com documentos formais sendo utilizados como meio de comunicação entre as etapas do processo. Por exemplo: os requisitos evoluirão e, no final das contas, será produzida uma especificação deles, que servirá como entrada para o processo de projeto e implementação. Em uma abordagem ágil, por outro lado, a iteração ocorre ao longo das atividades. Portanto, os requisitos e o projeto (*design*) são desenvolvidos juntos, e não separadamente.

FIGURA 3.1 Desenvolvimento dirigido por plano e desenvolvimento ágil.



Na prática, conforme explicarei na Seção 3.4.1, os processos dirigidos por plano são utilizados frequentemente com práticas de programação ágil, e os métodos ágeis podem incorporar algumas atividades planejadas, além da programação e dos testes. Em um processo dirigido por plano, é perfeitamente viável alocar requisitos e planejar a fase de projeto (*design*) e desenvolvimento como uma série de incrementos. Já um processo ágil não é inevitavelmente focado no código e pode produzir alguma documentação de projeto (*design*). Nos métodos ágeis, os desenvolvedores podem decidir que uma iteração não produzirá código novo, mas sim modelos e documentação de sistema.

3.1 MÉTODOS ÁGEIS

Nos anos 1980 e início dos anos 1990, havia uma percepção generalizada de que a maneira mais indicada para obter um software melhor era por meio do planejamento

cuidadoso do projeto, da garantia de qualidade formalizada, do uso de métodos de análise e projeto (*design*) apoiados por ferramentas de software e de processos de desenvolvimento controlados e rigorosos. Essa percepção veio da comunidade de engenharia de software que era responsável por desenvolver sistemas grandes e duradouros, como os destinados aos setores aeroespacial e governamental.

Essa abordagem dirigida por plano foi desenvolvida para o software criado por grandes times, trabalhando para diferentes empresas. Os times costumavam ficar dispersos geograficamente e trabalhavam no software por longos períodos. Um exemplo de software produzido dessa maneira são os sistemas de controle de uma aeronave moderna, que poderiam levar até dez anos para serem desenvolvidos, desde sua especificação inicial até sua implantação. As abordagens dirigidas por plano envolvem sobrecargas no planejamento, no desenvolvimento e na documentação do sistema. Essa sobrecarga é justificada quando o trabalho de vários times de desenvolvimento precisa ser coordenado, quando o sistema é crítico e quando muitas pessoas diferentes estarão envolvidas na manutenção do software ao longo de sua vida útil.

No entanto, quando essa abordagem pesada é aplicada a sistemas de negócio de pequeno ou médio porte, a sobrecarga é tão grande que domina o processo de desenvolvimento de software. Mais tempo é investido na decisão de como o sistema deve ser desenvolvido do que na programação ou nos testes. À medida que os requisitos mudam, retrabalho é necessário e, ao menos a princípio, sua especificação e seu projeto (*design*) têm de mudar.

A insatisfação com essas abordagens levou ao surgimento dos métodos ágeis no final da década de 1990. Eles permitiram que o time de desenvolvimento se concentrasse no próprio software, em vez de no projeto (*design*) ou na documentação. Os métodos ágeis são mais adequados para desenvolver aplicações nas quais os requisitos do sistema mudam rapidamente durante o processo. Eles se destinam a fornecer rapidamente um software funcional para o cliente, que, por sua vez, pode propor a inclusão de requisitos novos ou modificados nas iterações seguintes. Eles visam reduzir a burocracia do processo ao evitar o trabalho com valor duvidoso no longo prazo e eliminar a documentação que provavelmente jamais será utilizada.

A filosofia por trás dos métodos ágeis está refletida no 'manifesto ágil' (<http://agilemanifesto.org>), criado por desenvolvedores líderes desses métodos. O manifesto diz:

Estamos descobrindo maneiras melhores de desenvolver software, fazendo-o nós mesmos e ajudando outros a fazerem o mesmo. Através deste trabalho, passamos a valorizar:

Indivíduos e interações mais que processos e ferramentas.

Software em funcionamento mais que uma documentação abrangente.

Colaboração com o cliente mais que negociação de contratos.

Responder a mudanças mais que seguir um plano.

Ou seja, mesmo havendo valor nos itens à direita, valorizamos mais os itens à esquerda¹.

Todos os métodos ágeis sugerem que o software deve ser desenvolvido e entregue incrementalmente. Esses métodos se baseiam em diferentes processos ágeis, mas compartilham um conjunto de princípios, com base no manifesto ágil, e por isso têm muito em comum. Apresento esses princípios na Figura 3.2.

¹ Disponível em: <<http://agilemanifesto.org/iso/ptbr/manifesto.html>>. Acesso em: 30 mar 2018.

FIGURA 3.2 Os princípios dos métodos ágeis.

Princípio	Descrição
Envolvimento do cliente	Os clientes devem ser envolvidos em todo o processo de desenvolvimento. Seu papel é fornecer e priorizar novos requisitos de sistema e avaliar as iterações do sistema.
Acolher as mudanças	Tenha em mente que os requisitos do sistema mudam e, portanto, deve-se projetar o sistema para acomodar essas mudanças.
Entrega incremental	O software é desenvolvido em incrementos, e o cliente especifica os requisitos incluídos em cada um deles.
Manter a simplicidade	Deve-se ter como foco a simplicidade, tanto do software que está sendo desenvolvido quanto do processo de desenvolvimento. Sempre que possível, trabalhe ativamente para eliminar a complexidade do sistema.
Pessoas, não processos	As habilidades do time de desenvolvimento devem ser reconhecidas e aproveitadas da melhor maneira possível. Seus membros devem ter liberdade para desenvolver seu modo próprio de trabalhar sem se prender a processos determinados.

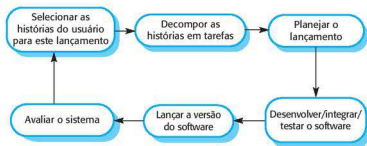
Os métodos ágeis têm sido particularmente úteis para dois tipos de desenvolvimento de sistemas:

1. O desenvolvimento de um produto pequeno ou médio, por uma empresa de software, para venda. Praticamente todos os produtos de software e aplicativos são desenvolvidos atualmente usando uma abordagem ágil.
2. O desenvolvimento de sistemas personalizados dentro de uma organização, em que há um compromisso claro por parte do cliente de se envolver no processo de desenvolvimento, e no qual há poucos *stakeholders* externos e normas que afetem o software.

Os métodos ágeis funcionam bem nessas situações porque possibilitam comunicação contínua entre o gerente de produto ou o cliente do sistema e o time de desenvolvimento. O software em si é um sistema *stand-alone*, em vez de estar intimamente integrado com outros sistemas que estejam sendo desenvolvidos simultaneamente. Consequentemente, não há necessidade de coordenar fluxos de desenvolvimento paralelos. Os sistemas pequenos e médios podem ser desenvolvidos por times situados no mesmo local, então a comunicação informal entre seus membros funciona bem.

3.2 TÉCNICAS DE DESENVOLVIMENTO ÁGIL

As ideias por trás dos métodos ágeis foram desenvolvidas mais ou menos na mesma época, nos anos 1990, por uma série de pessoas. Entretanto, talvez a abordagem mais importante para mudança de cultura no desenvolvimento de software tenha sido o desenvolvimento da Programação Extrema (do inglês *Extreme Programming*, ou XP). O nome foi cunhado por Kent Beck com base na ideia de que essa abordagem leva a níveis “extremos” boas práticas reconhecidas, como o desenvolvimento iterativo (BECK, 1998). Na Programação Extrema, por exemplo, várias versões novas de um sistema podem ser desenvolvidas — por diferentes programadores —, integradas e testadas em um dia. A Figura 3.3 ilustra o processo de produção do incremento de um sistema que está sendo desenvolvido usando Programação Extrema.

FIGURA 3.3 Ciclo de lançamento (*release*) da Programação Extrema.

Na XP, os requisitos são expressos em cenários (chamados de histórias do usuário) implementados diretamente como uma série de tarefas. Os programadores trabalham em pares e desenvolvem testes para cada tarefa antes de escreverem o código. Todos os testes devem ser executados com sucesso quando o novo código é integrado ao sistema, já que há um curto intervalo de tempo entre os lançamentos (*releases*) do sistema.

A Programação Extrema era controversa, pois introduziu uma série de práticas ágeis muito diferentes do desenvolvimento tradicional da época. Elas estão resumidas na Figura 3.4 e refletem os princípios do manifesto ágil:

1. O desenvolvimento incremental é apoiado por lançamentos menores e mais frequentes do sistema. Os requisitos se baseiam em histórias simples dos clientes — ou cenários —, utilizados como base para decidir qual funcionalidade deve ser incluída em um determinado incremento.
2. O envolvimento do cliente é apoiado por seu engajamento contínuo no time de desenvolvimento. O representante do cliente participa do desenvolvimento, e é responsável por definir os testes de aceitação do sistema.
3. As pessoas, não os processos, são apoiadas pela programação em pares, pela propriedade coletiva do código do sistema e por um processo de desenvolvimento sustentável que não envolve expedientes de trabalho longos demais.
4. As mudanças são adotadas por meio de lançamentos regulares do sistema aos clientes, desenvolvimento com testes *a priori* (*test-first*), refatoração para evitar a degeneração do código e integração contínua de novas funcionalidades.
5. A manutenção da simplicidade é apoiada pela refatoração constante, que melhora a qualidade do código, e pelo uso de projetos (*designs*) simples, que não antecipam desnecessariamente as futuras mudanças no sistema.

Na prática, a aplicação da Programação Extrema como proposta originalmente se provou mais difícil do que o previsto. Ela não pode ser integrada de imediato às práticas de gestão e à cultura da maioria das empresas; assim, as que adotam métodos ágeis selecionam as práticas de Programação Extrema mais adequadas ao seu modo de trabalho. Às vezes, essas técnicas são incorporadas aos processos de desenvolvimento das próprias empresas, mas, na maioria das vezes, são utilizadas em conjunto com um método ágil focado em gerenciamento, como o Scrum (RUBIN, 2013).

FIGURA 3.4 Práticas de programação Extrema.

Princípio ou prática	Descrição
Propriedade coletiva	Os pares de desenvolvedores trabalham em todas as áreas do sistema de modo que não se desenvolvem 'ilhas de conhecimento', e todos os desenvolvedores assumem a responsabilidade por todo o código. Qualquer um pode mudar qualquer coisa.
Integração contínua	Assim que o trabalho em uma tarefa é concluído, ele é integrado ao sistema completo. Após qualquer integração desse tipo, todos os testes de unidade no sistema devem passar.
Planejamento incremental	Os requisitos são registrados em 'cartões de história', e as histórias a serem incluídas em um lançamento são determinadas de acordo com o tempo disponível e com sua prioridade relativa. Os desenvolvedores decompõem essas histórias em 'tarefas' de desenvolvimento (Figuras 3.5 e 3.6).
Representante do cliente	Um representante do usuário final do sistema (o cliente) deve estar disponível em tempo integral para o time de programação. Em um processo como esse, o cliente é um membro do time de desenvolvimento, sendo responsável por levar os requisitos do sistema ao time, visando sua implementação.
Programação em pares	Os desenvolvedores trabalham em pares, conferindo o trabalho um do outro e oferecendo o apoio necessário para que o resultado final seja sempre satisfatório.
Refatoração	Todos os desenvolvedores devem refatorar o código continuamente logo que sejam encontradas possíveis melhorias para ele. Isso mantém o código simples e de fácil manutenção.
Projeto (<i>design</i>) simples	Deve ser feito o suficiente de projeto (<i>design</i>) para satisfazer os requisitos atuais, e nada mais.
Lançamentos pequenos	O mínimo conjunto útil de funcionalidade que agregue valor ao negócio é desenvolvido em primeiro lugar. Os lançamentos do sistema são frequentes e acrescentam funcionalidade à primeira versão de uma maneira incremental.
Ritmo sustentável	Grandes quantidades de horas extras não são consideradas aceitáveis, já que o efeito líquido muitas vezes é a diminuição da qualidade do código e da produtividade no médio prazo.
Desenvolvimento com testes <i>a priori</i> (<i>test-first</i>)	Um <i>framework</i> automatizado de teste de unidade é utilizado para escrever os testes de um novo pedaço de funcionalidade antes que ela própria seja implementada.

Não estou convencido de que a Programação Extrema seja um método ágil prático para a maioria das empresas, mas sua contribuição mais importante é, provavelmente, o conjunto de práticas de desenvolvimento ágil que introduziu na comunidade. Discuto as mais importantes nesta seção.

3.2.1 Histórias do usuário

Os requisitos de software sempre mudam. Para lidar com essas mudanças, os métodos ágeis não têm uma atividade de engenharia de requisitos específica ou independente. Em vez disso, a elicitación dos requisitos é integrada ao desenvolvimento. Para facilitar esse processo, foi desenvolvida a ideia de 'histórias do usuário', com o intuito de formar cenários de uso baseados nas experiências de um usuário do sistema.

Na medida do possível, o cliente do sistema trabalha em estreita colaboração com o time de desenvolvimento e discute esses cenários com os membros do time. Juntos, eles desenvolvem um 'cartão de história' que descreve resumidamente uma história que reúna as necessidades do usuário. O time de desenvolvimento buscará, então, implementar esse cenário em uma versão futura do software. Um exemplo de cartão de história do sistema Mentcare é exibido na Figura 3.5. Trata-se de uma descrição sucinta de um cenário para prescrever medicação para um paciente.

FIGURA 3.5 Uma história sobre prescrição de medicação.**Prescrição de medicação**

Kate é uma médica que deseja prescrever medicação para um paciente que frequenta sua clínica. O registro do paciente já é exibido em seu computador, então ela deve clicar no campo de medicação e selecionar 'medicação atual', 'nova medicação' ou 'substâncias'.

Se escolher 'medicação atual', o sistema pedirá para que ela verifique a dose; se quiser mudá-la, Kate fornecerá a nova dose e depois deverá confirmar a prescrição.

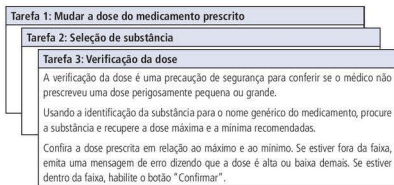
Se escolher 'nova medicação', o sistema presumirá que ela sabe qual medicação prescrever. Ao digitar as primeiras letras da medicação, o sistema exibirá uma lista de possíveis medicamentos que começam com essas letras. Ao escolher a medicação necessária, o sistema responderá pedindo que ela confirme se a medicação escolhida está correta. Ela deverá fornecer a dose e depois confirmar a prescrição.

Se ela escolher 'substâncias', o sistema exibirá uma caixa de busca para a substância aprovada, e então ela poderá pesquisar o medicamento que deseja. Feita a busca, lhe será solicitado que confirme se o medicamento selecionado está correto. Ela deverá fornecer a dose e depois confirmar a prescrição.

O sistema sempre verificará se a dose está dentro da faixa aprovada. Se não estiver, será pedido a Kate que faça uma alteração.

Depois de confirmar a prescrição, ela será exibida para conferência. Kate deverá clicar em 'OK' ou em 'Alterar'. Se 'OK' for selecionado, a prescrição será registrada no banco de dados de auditoria. Se clicar em 'Alterar', o processo de 'Prescrição de medicação' é executado novamente.

As histórias do usuário podem ser utilizadas no planejamento das iterações do sistema. Depois de desenvolvidos, os cartões de história devem ser decompostos em tarefas pelo time de desenvolvimento (Figura 3.6), que estima o esforço e os recursos necessários para implementar cada uma delas. Normalmente, isso envolve discussões com o cliente para refinar os requisitos. O cliente prioriza as histórias a serem implementadas, escolhendo as que podem ser utilizadas imediatamente para proporcionar suporte útil ao negócio. A intenção é identificar funcionalidades essenciais que possam ser implementadas em aproximadamente duas semanas, quando a próxima versão do sistema é disponibilizada para o cliente.

FIGURA 3.6 Exemplos de cartões de tarefa para prescrição de medicação.

Naturalmente, à medida que os requisitos mudam, as histórias não implementadas também mudam ou são descartadas. Se forem necessárias alterações em um sistema que já foi entregue, são elaborados novos cartões de história e, mais uma vez, o cliente decide se essas alterações devem ter ou não prioridade sobre novas funcionalidades.

A ideia de histórias do usuário é poderosa, e as pessoas acham muito mais fácil se identificar com elas do que com o documento convencional de requisitos ou com casos de uso. As histórias podem ser úteis para fazer com que os usuários se envolvam na sugestão de requisitos durante uma atividade de elicitação que anteceda o desenvolvimento. Discutirei isso em mais detalhes no Capítulo 4.

Por outro lado, o problema principal com as histórias do usuário é a completude. É difícil julgar se foram desenvolvidas histórias suficientes para cobrir todos os requisitos essenciais de um sistema. Também é difícil julgar se uma única história proporciona uma imagem completa de uma atividade. Usuários mais experientes estão frequentemente tão familiarizados com seu trabalho que deixam de fora algumas coisas quando o descrevem.

3.2.2 Refatoração

Uma premissa fundamental da engenharia de software tradicional é a de que se deve projetar com vistas à mudança; ou seja, deve-se prever as futuras mudanças no software e projetá-lo para que elas possam ser implementadas com facilidade. No entanto, a Programação Extrema descartou esse princípio, com base na ideia de que projetar já pensando na mudança leva a um desperdício de trabalho. Não vale a pena perder tempo acrescentando generalidades a um programa para lidar com a mudança. Frequentemente, as alterações previstas não se materializam ou as solicitações de mudança podem ser completamente diferentes do previsto.

É natural que, na prática, sempre haja modificações no código que está sendo desenvolvido. Para facilitar esse processo, os desenvolvedores da Programação Extrema sugeriram que o código em desenvolvimento seja refatorado constantemente. ‘Refatorar’ (FOWLER *et al.*, 1999) significa que o time de programação deve buscar possíveis melhorias no software e implementá-las imediatamente. Quando os membros do time se deparam com um código que pode ser melhorado, eles executam essas melhorias, mesmo quando não há necessidade imediata delas.

Um problema fundamental do desenvolvimento incremental é que as mudanças tendem a degradar a estrutura do software; consequentemente, fica cada vez mais difícil realizar outras alterações. Essencialmente, o desenvolvimento prossegue conforme são encontradas soluções alternativas para os problemas, o que resulta na frequente duplicação de código, no reúso indevido de partes do software e na degradação geral da estrutura à medida que código é adicionado ao sistema. A refatoração melhora a estrutura do software e a sua clareza, evitando, com isso, a deterioração estrutural que ocorre naturalmente quando o software é modificado.

Exemplos de refatoração incluem a reorganização de uma hierarquia de classe para remover código duplicado, a organização e renomeação de atributos e métodos e a substituição de seções de código similares, com chamadas para métodos definidos em uma biblioteca. Os ambientes de desenvolvimento de programas geralmente incluem ferramentas para refatoração. Essas ferramentas tornam mais fácil o processo de encontrar dependências entre as seções e de fazer modificações globais no código.

A princípio, quando a refatoração faz parte do processo de desenvolvimento, o software sempre deve ser fácil de compreender e mudar quando são propostos novos requisitos. Na prática, nem sempre é o que acontece. Às vezes, a pressão do desenvolvimento significa que a refatoração é postergada em virtude do tempo dedicado à implementação da nova funcionalidade. Algumas características novas e alterações não podem ser prontamente absorvidas pela refatoração de código e requerem que a arquitetura do sistema seja modificada.

3.2.3 Desenvolvimento com testes *a priori* (test-first)

Conforme discuti na introdução deste capítulo, uma das diferenças importantes entre o desenvolvimento incremental e o desenvolvimento dirigido por plano é a maneira como o sistema é testado. Com o desenvolvimento incremental, não há uma especificação do sistema que possa ser utilizada por uma equipe de testes externa para desenvolver testes do sistema. Como consequência, algumas abordagens ao desenvolvimento incremental têm um processo de testes muito informal em comparação com os testes do desenvolvimento dirigido por plano.

A Programação Extrema desenvolveu uma nova abordagem ao teste de programas para contornar as dificuldades de se testar sem uma especificação. O teste é automatizado e é fundamental para o processo de desenvolvimento, que não consegue avançar até que todos os testes tenham sido executados com sucesso. As características-chave do teste em Programação Extrema são:

1. desenvolvimento com testes *a priori* (test-first);
2. desenvolvimento de teste incremental a partir de cenários;
3. envolvimento do usuário no desenvolvimento e validação dos testes e;
4. o uso de *frameworks* de teste automatizados.

Hoje, a filosofia de testes *a priori* (test-first) da Programação Extrema evoluiu para técnicas de desenvolvimento dirigido por testes mais gerais (JEFFRIES; MELNIK, 2007). Acredito que o desenvolvimento dirigido por testes é uma das inovações mais importantes na engenharia de software. Em vez de escrever o código e depois os testes dele, primeiro se escrevem os testes e depois o código. Isso significa que é possível executar os testes à medida que o código está sendo escrito e descobrir problemas durante o desenvolvimento. Discutirei o desenvolvimento dirigido por testes com mais profundidade no Capítulo 8.

Escrever os testes define implicitamente uma interface e uma especificação do comportamento da funcionalidade que está sendo desenvolvida. Os problemas de requisitos e má compreensão da interface são menores. O desenvolvimento com testes *a priori* exige que haja uma relação clara entre os requisitos do sistema e o código que implementa os requisitos correspondentes. Na Programação Extrema, essa relação é clara porque os cartões de história que representam os requisitos são decompostos em tarefas, e essas são a unidade principal de implementação.

No desenvolvimento com testes *a priori*, as pessoas que implementam as tarefas precisam compreender inteiramente a especificação para que sejam capazes de escrever os testes do sistema. Isso significa que as ambiguidades e omissões na especificação devem ser esclarecidas antes do início da implementação. Além disso, ele também evita o problema de 'defasagem do teste', algo que pode acontecer quando o desenvolvedor do sistema trabalha em um ritmo mais veloz que o do testador. A implementação fica cada vez mais à frente e há uma tendência a ignorar testes para que o cronograma de desenvolvimento possa ser mantido.

A abordagem com testes *a priori* da Programação Extrema pressupõe que as histórias do usuário foram desenvolvidas e decompostas em um conjunto de cartões de tarefa, conforme a Figura 3.6. Cada tarefa gera um ou mais testes de unidade que conferem a implementação descrita na tarefa. A Figura 3.7 é uma descrição curta de um caso de teste que foi desenvolvido para conferir se a dose prescrita de um medicamento não está fora dos limites de segurança conhecidos.

FIGURA 3.7 Descrição de caso de teste para conferir a dose.**Teste 4: Conferir a dose****Entrada:**

1. Um número em mg representando uma única dose de medicamento.
2. Um número representando a quantidade de doses únicas por dia.

Testes:

1. Teste das entradas em que a dose única está correta, mas a frequência é alta demais.
2. Teste das entradas em que a dose única é alta demais e baixa demais.
3. Teste das entradas em que a dose única * frequência é alta demais e baixa demais.
4. Teste das entradas em que a dose única * frequência está no intervalo permitido.

Saída:

'OK' ou mensagem de erro indicando que a dose está fora do intervalo seguro.

O papel do cliente nesse processo é ajudar a desenvolver testes de aceitação para as histórias que devem ser implementadas no próximo lançamento do sistema. Conforme explico no Capítulo 8, o teste de aceitação é o processo pelo qual o sistema é testado usando dados do cliente, com o objetivo de conferir se as suas reais necessidades estão sendo satisfeitas.

A automação dos testes é essencial para o desenvolvimento com testes *a priori* (*test-first*). Os testes são escritos como componentes executáveis antes de a tarefa ser implementada. Esses componentes de teste devem ser *stand-alone*, simular a submissão da entrada a ser testada e conferir se o resultado satisfaz a especificação de saída. Um *framework* de teste automatizado é um sistema que facilita a produção de testes executáveis e os submete à execução. O JUnit (TAHCHIEV *et al.*, 2010) é um exemplo de *framework* de teste automatizado amplamente utilizado para programas em Java.

Como os testes são automatizados, há sempre um conjunto deles que pode ser executado de maneira rápida e fácil. Sempre que qualquer funcionalidade for acrescentada ao sistema, os testes podem ser rodados e os problemas que o novo código introduziu podem ser capturados imediatamente.

O desenvolvimento com testes *a priori* e o teste automatizado resultam normalmente em uma grande quantidade de testes sendo produzidos e executados simultaneamente. No entanto, existem problemas ao garantir que a cobertura dos testes seja completa:

1. Os programadores preferem programar a testar e, às vezes, tomam atalhos quando elaboram os testes. Por exemplo, eles podem escrevê-los incompletos, sem a conferência de todas as possíveis exceções que podem ocorrer.
2. Alguns testes podem ser muito difíceis de escrever de modo incremental. Por exemplo, em uma interface com o usuário complexa, muitas vezes é difícil escrever testes de unidade para o código que implementa a 'lógica de exibição' e o fluxo entre as telas.

É difícil julgar se um conjunto de testes é ou não completo. Embora seja possível ter muitos testes do sistema, seu conjunto pode não promover a cobertura completa. Partes cruciais do sistema podem não ser executadas e, assim, permanecerão não testadas. Portanto, embora um grande conjunto de testes executados frequentemente possa dar a impressão de que o sistema está completo e correto, isso pode não ser verdade. Se não for feita uma revisão e outros testes forem produzidos após o desenvolvimento, então os erros não detectados podem seguir junto com o lançamento do sistema.

3.2.4 Programação em pares

Outra iniciativa prática que foi introduzida na Programação Extrema é que os programadores trabalham em pares no desenvolvimento do software. Cada dupla senta-se diante do mesmo computador para desenvolver o software, mas o mesmo par nem sempre programa junto. Em vez disso, os pares são criados dinamicamente para que todos os membros do time trabalhem uns com os outros durante o processo de desenvolvimento.

A programação em pares tem uma série de vantagens:

1. Apóia a ideia de propriedade e responsabilidade coletivas pelo sistema. Isso reflete a ideia de Weinberg, de programação sem ego (WEINBERG, 1971), em que o software é de propriedade de todo o time e os indivíduos não são responsabilizados individualmente pelos problemas com o código. Em vez disso, todos são responsáveis pela resolução de problemas.
2. Ela age como um processo de revisão informal, já que cada linha de código é examinada por ao menos duas pessoas. Essas inspeções e revisões (Capítulo 24) são eficazes na descoberta de um alto percentual de erros de software, mas é preciso muito tempo para organizá-las e normalmente resultam em atrasos no processo de desenvolvimento. A programação em pares é um processo menos formal, que provavelmente não encontra tantos erros quanto as inspeções de código, mas é mais barata e fácil de organizar do que as inspeções formais.
3. Incentiva a refatoração para melhorar a estrutura do software. O problema em pedir aos programadores para refatorarem em um ambiente de desenvolvimento normal é que o esforço envolvido é investido para obter benefício no longo prazo. Um desenvolvedor que investe tempo refatorando pode ser considerado menos eficiente do que um que simplesmente desenvolve código. Nas situações em que a programação em pares e a propriedade coletiva são empregadas, outras pessoas se beneficiam imediatamente da refatoração, tendendo a apoiarem o processo.

Pode-se pensar que a programação em pares é menos eficiente do que a programação individual. Em um determinado intervalo de tempo, um par de desenvolvedores produziria a metade do código produzido por dois indivíduos trabalhando sozinhos. Muitas empresas que adotaram métodos ágeis suspeitam da programação em pares e não a utilizam. Outras misturam a programação em pares e a individual, com um programador experiente trabalhando com um colega menos experiente quando encontram problemas.

Estudos formais sobre o valor da programação em pares obtiveram resultados diversos. Usando estudantes como voluntários, Williams e seus colaboradores (WILLIAMS *et al.*, 2000) constataram que a produtividade da programação em pares parece ser comparável à de duas pessoas trabalhando de maneira independente. Os motivos sugeridos são que os pares discutem o software antes do desenvolvimento e, portanto, provavelmente têm menos partidas em falso e menos retrabalho. Além disso, o número de erros evitados pela inspeção informal é tal que menos tempo é consumido consertando defeitos descobertos durante o processo de teste.

Entretanto, estudos com programadores mais experientes não reproduzem esses resultados (ARISHOLM *et al.*, 2007). Eles constataram que havia uma perda

significativa de produtividade se comparado a dois programadores trabalhando isoladamente. Houve alguns benefícios para a qualidade, mas não compensaram totalmente a sobrecarga da programação em pares. Todavia, o compartilhamento de conhecimento que acontece durante a programação em pares é muito importante, já que reduz os riscos globais para um projeto quando os membros do time têm de deixá-lo. Isso já justifica, por si só, a aplicação da programação em pares.

3.3 GERENCIAMENTO ÁGIL DE PROJETOS

Em qualquer empresa de software, os gerentes precisam saber o que está acontecendo e se um projeto tende ou não a cumprir seus objetivos, entregar o software no prazo e dentro do orçamento proposto. As abordagens dirigidas por plano evoluíram para satisfazer essa necessidade. Como discuto no Capítulo 23, os gerentes traçam um plano para o projeto mostrando o que deve ser entregue, quando deve ser entregue e quem vai trabalhar no desenvolvimento dos entregáveis desse projeto. Uma abordagem dirigida por plano exige que um gerente tenha uma visão estável sobre tudo que deve ser desenvolvido e sobre os processos de desenvolvimento.

O planejamento informal e o controle de projeto propostos pelos primeiros adeptos dos métodos ágeis conflitaram com essas necessidades de visibilidade impostas pelas empresas. Os times se auto-organizavam, não produziam documentação e planejavam o desenvolvimento em ciclos muito curtos. Embora isso possa funcionar (e funcione) nas empresas pequenas que desenvolvem produtos de software, é inadequado para as grandes empresas que precisam saber o que está acontecendo em sua organização.

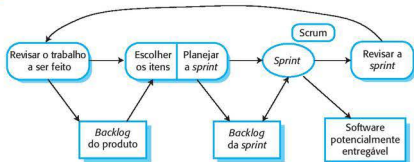
Assim como em todos os demais processos profissionais de desenvolvimento de software, o desenvolvimento ágil precisa ser gerenciado para que seja feito o melhor uso do tempo e dos recursos disponíveis para o time. Para tratar dessa questão, foi desenvolvido o método ágil chamado Scrum (SCHWABER; BEEDLE, 2001; RUBIN, 2013), a fim de proporcionar um arcabouço para organizar os projetos ágeis e, ao menos até certo ponto, dar visibilidade externa do que está acontecendo. Os desenvolvedores do Scrum queriam deixar claro que ele não era um método para gerenciamento de projetos no sentido convencional, então deliberadamente inventaram uma nova terminologia — como *Scrum Master* —, que substituiu nomes como o do gerente de projeto. A Figura 3.8 resume a terminologia do Scrum e o que ela significa.

O Scrum é um método ágil na medida em que segue os princípios do manifesto ágil, que mostrei na Figura 3.2. No entanto, ele se concentra em proporcionar um arcabouço para a organização ágil do projeto e não impõe o uso de práticas de desenvolvimento específicas, como a programação em pares e o desenvolvimento com testes *a priori* (*test-first*). Isso significa que ele pode ser integrado mais facilmente à prática atual de uma empresa. Consequentemente, à medida que os métodos ágeis se tornaram a principal abordagem para o desenvolvimento de software, o Scrum emergiu como o método mais utilizado.

O processo do Scrum ou ciclo da *sprint* é exibido na Figura 3.9. A entrada é o *backlog* do produto e cada iteração do processo gera um incremento do produto que poderia ser entregue para os clientes.

FIGURA 3.8 Terminologia do Scrum.

Termo do Scrum	Definição
Time de desenvolvimento	Um grupo auto-organizado de desenvolvedores de software que não deve ter mais de sete pessoas. Elas são responsáveis por desenvolver o software e outros documentos essenciais do projeto.
Incremento de produto potencialmente entregável	O incremento de software entregue a partir de uma <i>sprint</i> . A ideia é que ele seja 'potencialmente entregável', significando que está em estado acabado e não é necessário um trabalho adicional, como testes, para incorporá-lo ao produto final. Na prática, contudo, nem sempre isso é realizável.
Backlog do produto	É uma lista de itens 'a fazer' que o time Scrum deve cumprir. Podem ser definições de características e requisitos do software, histórias do usuário ou descrições de tarefas suplementares que são necessárias, como a definição da arquitetura ou a documentação do usuário.
Product Owner	Um indivíduo (ou possivelmente um pequeno grupo) cujo dever é identificar características ou requisitos do produto, priorizá-los para desenvolvimento e revisar continuamente o <i>backlog</i> do produto para garantir que o projeto continue a satisfazer as necessidades críticas do negócio. O <i>Product Owner</i> , também chamado de dono do produto, pode ser um cliente, mas também poderia ser um gerente de produto em uma empresa de software ou um representante de um <i>stakeholder</i> .
Scrum	Uma reunião diária do time Scrum que examina o progresso e prioriza o trabalho a ser feito naquele dia. Em condições ideais, deve ser uma reunião presencial que inclua todo o time.
Scrum Master	O <i>Scrum Master</i> é responsável por assegurar que o processo Scrum seja seguido e guiar o time no uso eficaz do Scrum. Essa pessoa é responsável pela interação com o resto da empresa e por garantir que o time Scrum não seja desviado por interferências externas. Os desenvolvedores Scrum são inflexíveis quanto ao <i>Scrum Master</i> não ser considerado um gerente de projeto. No entanto, outros nem sempre podem ver a diferença facilmente.
Sprint	Uma iteração de desenvolvimento. As <i>sprints</i> normalmente duram de 2 a 4 semanas.
Velocidade	Uma estimativa de quanto esforço do <i>backlog</i> do produto um time pode cobrir em uma única <i>sprint</i> . Compreender a velocidade de um time ajuda a estimar o que pode ser coberto por uma <i>sprint</i> e constitui a base para medir a melhoria do desempenho.

FIGURA 3.9 O ciclo de *sprint* do Scrum.

O ponto de partida para o ciclo da *sprint* do Scrum é o *backlog* do produto – uma lista de itens como características do produto, requisitos e melhorias de engenharia que precisam ser trabalhados pelo time Scrum. A versão inicial desse *backlog* pode ser derivada de um documento de requisitos, uma lista de histórias do usuário ou outra descrição do software a ser desenvolvido.

Enquanto a maioria das entradas no *backlog* do produto se preocupa com a implementação das características do sistema, outras atividades também podem ser incluídas. Às vezes, durante o planejamento de uma iteração, aparecem questões que não podem ser respondidas com facilidade, sendo necessário mais trabalho para explorar as possíveis soluções. O time pode fazer alguma prototipação ou desenvolvimento experimental para entender o problema e a solução. Também pode haver itens do *backlog* para projetar a arquitetura do sistema ou desenvolver a sua documentação.

O *backlog* do produto pode ser especificado em vários níveis de detalhe, sendo responsabilidade do *Product Owner*, ou dono do produto, garantir que o nível de detalhe da especificação seja adequado ao trabalho a ser realizado. Por exemplo, um item de *backlog* poderia ser uma história de usuário completa, como mostra a Figura 3.5, ou ser simplesmente uma instrução, como “refatorar o código da interface com o usuário”, deixando para o time a decisão sobre a refatoração a ser feita.

Cada ciclo da *sprint* dura um intervalo de tempo fixo, que normalmente é de 2 a 4 semanas. No início de cada ciclo, o *Product Owner* prioriza os itens no *backlog* do produto para definir quais são os mais importantes a serem desenvolvidos naquele ciclo. As *sprints* nunca se estendem para levar em conta trabalhos inacabados. Os itens são devolvidos para o *backlog* do produto se não puderem ser concluídos dentro do tempo alocado para a *sprint*.

Depois, todo o time se envolve na escolha dos itens de mais alta prioridade que acreditam que devam ser concluídos e estimam o tempo necessário para concluí-los. Para fazer essas estimativas, eles usam a velocidade alcançada nas *sprints* anteriores, ou seja, quanto do *backlog* pôde ser concluído em uma única *sprint*. Isso leva à criação de um *backlog* da *sprint*—o trabalho a ser feito durante a *sprint*. O time se auto-organiza para decidir quem irá trabalhar em que antes de começar.

Durante esse tempo, o time faz reuniões diárias curtas (Scrums) para revisar o progresso e, onde for necessário, alterar as prioridades do trabalho. Durante o Scrum, todos os membros do time compartilham informações, descrevem seu progresso desde a última reunião, trazem os problemas que surgiram e declaram o que foi planejado para o dia seguinte. Desse modo, todos no time sabem o que está acontecendo e, se surgirem problemas, podem planejar novamente o trabalho no curto prazo para lidar com eles. Todos participam desse planejamento e não há uma orientação de cima para baixo partindo do *Scrum Master*.

As interações diárias entre os times podem ser coordenadas usando um quadro do Scrum. É um tipo de lousa de escritório que inclui informações e notas em *post-it* sobre o *backlog* da *sprint*: o trabalho realizado, a indisponibilidade do time etc. Esse recurso é compartilhado e todos podem modificar ou mover os itens na lousa. Isso significa que qualquer membro do time pode ver, de relance, o que os outros estão fazendo e qual é o trabalho que ainda não foi feito.

No final de cada *sprint* há uma reunião para revisão que envolve todo o time e possui duas finalidades: em primeiro lugar, é um meio de melhorar processos, já que o time examina seu modo de trabalho e reflete sobre como as coisas poderiam ter sido feitas de uma maneira melhor; em segundo lugar, fornece insumos sobre o produto e seu estado para a revisão do *backlog* do produto que precede a próxima *sprint*.

Embora o *Scrum Master* não seja formalmente um gerente de projeto, ele, na prática, assume esse papel em muitas organizações que possuem uma estrutura de gestão convencional. Eles reportam o progresso para a alta gerência e estão envolvidos no planejamento de longo prazo e no orçamento do projeto. Também podem se envolver na administração do projeto (chegando a um consenso com o time a respeito dos feriados, mantendo contato com o RH etc.) e nas aquisições de hardware e software.

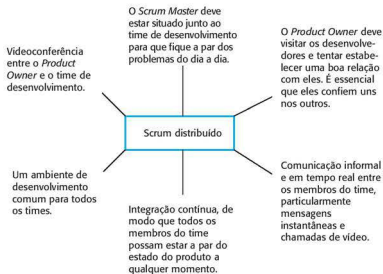
Em várias histórias de sucesso do Scrum (SCHATZ; ABDELSHAFI, 2005; MULDER; VAN VLIET, 2008; BELLOUITI, 2009), as coisas que os usuários mais gostam a respeito do método são:

1. O produto é decomposto em um conjunto de “pedaços” gerenciáveis e compreensíveis aos quais os *stakeholders* podem se referir.
2. Requisitos instáveis não impedem o progresso.
3. O time inteiro tem visibilidade de tudo e, conseqüentemente, a comunicação e disposição de seus membros são melhores.
4. Os clientes veem a entrega dos incrementos na hora e obtêm *feedback* de como o produto funciona. Eles não se deparam com surpresas de última hora como quando um time anuncia que o software não será entregue conforme o esperado.
5. A confiança entre clientes e desenvolvedores é estabelecida, criando uma cultura positiva na qual todos esperam que o projeto tenha sucesso.

O Scrum, como foi projetado originalmente, destinava-se ao uso por times situados num mesmo local, onde todos os envolvidos pudessem se reunir diariamente. No entanto, hoje, muito do desenvolvimento de software envolve times distribuídos, com membros situados em diferentes lugares no mundo. Isso permite que as empresas tirem proveito de mão de obra mais barata em outros países, possibilita o acesso a habilidades especializadas e permite o desenvolvimento 24 horas por dia, com o trabalho acontecendo em diferentes fusos horários.

Conseqüentemente, tem havido desdobramentos do Scrum para ambientes de desenvolvimento distribuídos e para o trabalho de múltiplos times. Normalmente, no desenvolvimento geograficamente distribuído, o *Product Owner* está em um país diferente do país do time de desenvolvimento, que também pode ser distribuído. A Figura 3.10 mostra os requisitos do Scrum distribuído (DEEMER, 2011).

FIGURA 3.10 Scrum distribuído.



3.4 ESCALABILIDADE DOS MÉTODOS ÁGEIS

Os métodos ágeis foram desenvolvidos para serem utilizados por pequenos times de programação que poderiam trabalhar juntos na mesma sala e se comunicar informalmente. Eram utilizados originalmente no desenvolvimento de sistemas e produtos

de software de pequeno e médio porte. Empresas pequenas, sem processos formais ou burocracia, foram os primeiros usuários entusiastas desses métodos.

Naturalmente, a necessidade de uma entrega mais rápida do software, que seja mais adequada às necessidades do cliente, também se aplica aos sistemas e empresas maiores. Consequentemente, ao longo dos últimos anos, muito trabalho tem sido feito no desenvolvimento dos métodos ágeis para sistemas de software grandes e para uso em grandes empresas.

A escalabilidade dos métodos ágeis tem duas facetas claramente relacionadas:

1. escalar verticalmente (*scaling up*) esses métodos para lidar com o desenvolvimento de sistemas grandes demais para serem assumidos por um único time pequeno;
2. escalar horizontalmente (*scaling out*) esses métodos dos times de desenvolvedores especializados para o uso disseminado em uma grande empresa com muitos anos de experiência em desenvolvimento de software.

Naturalmente, escalar verticalmente (*scaling up*) e horizontalmente (*scaling out*) estão intimamente relacionados. Contratos para desenvolver sistemas de software grandes normalmente são concedidos às grandes organizações, com vários times trabalhando no projeto. Essas empresas costumam ter experiência em métodos ágeis aplicados a projetos menores, então enfrentam, ao mesmo tempo, problemas de escalar verticalmente (*scaling up*) e horizontalmente (*scaling out*).

Existem muitas histórias a respeito da eficácia dos métodos ágeis, e sugere-se que eles podem proporcionar melhorias significativas de produtividade e reduções comparáveis nos defeitos. Ambler (2010), um desenvolvedor de métodos ágeis influente, sugere que essas melhorias na produtividade são exageradas para os sistemas e organizações grandes. Ele sugere que uma organização que passe a usar tais métodos pode esperar uma melhoria de aproximadamente 15% na sua produtividade em um período de três anos, com reduções similares no número de defeitos do produto.

3.4.1 Problemas práticos dos métodos ágeis

Em algumas áreas, particularmente no desenvolvimento de produtos de software e aplicativos, o desenvolvimento ágil tem sido incrivelmente bem-sucedido. É de longe a melhor abordagem para esse tipo de sistema. Entretanto, esses métodos podem não ser adequados para outros tipos de desenvolvimento de software, como a engenharia de sistemas embarcados ou o desenvolvimento de sistemas grandes e complexos.

Nos sistemas grandes e de longa vida útil desenvolvidos por uma empresa de software para um cliente externo, o uso de uma abordagem ágil apresenta uma série de problemas:

1. A informalidade do desenvolvimento ágil é incompatível com a abordagem legal normalmente utilizada para a definição dos contratos em grandes empresas.
2. Os métodos ágeis são mais adequados para o desenvolvimento de software novo em vez de manutenção de software. Contudo, a maioria dos custos de software nas grandes empresas é proveniente da manutenção dos sistemas existentes.
3. Esses métodos são concebidos para pequenas empresas que compartilham uma localização geográfica, ainda que muito do desenvolvimento de software envolva, hoje em dia, times distribuídos mundialmente.

As questões contratuais podem ser um grande problema quando são utilizados os métodos ágeis. Quando o cliente usa uma organização externa para desenvolver seu sistema, é firmado um contrato entre elas. Normalmente, o documento de requisitos de software faz parte desse contrato entre o cliente e o fornecedor. Como o desenvolvimento intercalado dos requisitos e do código é fundamental para os métodos ágeis, não há uma declaração definitiva dos requisitos que possa ser incluída no contrato.

Consequentemente, os métodos ágeis devem se apoiar nos contratos em que o cliente paga pelo tempo necessário para o desenvolvimento do sistema em vez de pagar por um conjunto específico de requisitos. Contanto que tudo corra bem, isso traz benefícios tanto para o cliente quanto para o desenvolvedor. No entanto, se surgirem problemas, então pode haver controvérsias complicadas quanto a quem é o culpado e quem deveria pagar pelo tempo e recursos extras necessários para resolver os problemas.

Conforme explico no Capítulo 9, uma enorme quantidade de esforço de engenharia de software é dedicada à manutenção e evolução dos sistemas de software existentes. As práticas ágeis — como a entrega incremental, o projeto (*design*) para a mudança e a manutenção da simplicidade — fazem sentido quando o software está sendo modificado. Na verdade, é possível pensar em um processo de desenvolvimento ágil como um processo que apoia a mudança contínua. Se forem utilizados métodos ágeis para o desenvolvimento de produtos de software, novos lançamentos (*releases*) do produto ou aplicativo envolvem simplesmente a continuação da abordagem ágil.

No entanto, nas situações em que a manutenção envolve um sistema personalizado que deve ser modificado em resposta a novos requisitos do negócio, não há um consenso claro sobre a adequabilidade dos métodos ágeis para manutenção de software (BIRD, 2011; KILNER, 2012). Podem, então, surgir três tipos de problemas:

- ▶ falta de documentação do produto;
- ▶ dificuldade em manter os clientes envolvidos;
- ▶ continuidade do time de desenvolvimento.

A documentação formal deve descrever o sistema e, portanto, facilitar a compreensão das pessoas que o estão alterando. Porém, na prática, essa documentação raramente é atualizada e deixa de refletir com precisão o código do programa. Por essa razão, os entusiastas dos métodos ágeis argumentam que é uma perda de tempo escrevê-la e que a chave para implementar software passível de manutenção é produzir código de alta qualidade e legível. A falta de documentação não deve ser um problema nesse processo de manutenção com o uso de uma abordagem ágil.

No entanto, minha experiência em manutenção de sistemas mostra que o documento mais importante é o que descreve os requisitos do sistema e que diz ao engenheiro de software o que o sistema deve fazer. Sem esse tipo de conhecimento, é difícil avaliar o impacto das alterações propostas. Muitos métodos ágeis coletam os requisitos informalmente e de modo incremental, sem criar um documento coerente. Portanto, o uso desses métodos pode dificultar e encarecer a manutenção subsequente do sistema. Particularmente, isso é um problema se a continuidade do time de desenvolvimento não puder ser mantida.

Um desafio-chave no uso de uma abordagem ágil para a manutenção é manter os clientes envolvidos no processo. Embora eles possam justificar o envolvimento em tempo integral de um representante durante o desenvolvimento do sistema, isso é menos provável durante a manutenção, quando as mudanças não são contínuas. Os representantes do cliente tendem a perder o interesse no sistema.

Portanto, é provável que mecanismos alternativos, como as propostas de mudança — discutidas no Capítulo 25 —, venham a ser adaptados para se encaixar em uma abordagem ágil.

Outro possível problema é manter a continuidade do time de desenvolvimento. Os métodos ágeis exigem que os membros do time compreendam os aspectos do sistema sem que seja preciso consultar uma documentação. Se um time de desenvolvimento ágil for modificado, então esse conhecimento implícito se perde e é difícil para os novos membros terem a mesma compreensão do sistema e seus componentes. Muitos programadores preferem trabalhar no desenvolvimento de um novo software do que em sua manutenção e, assim, hesitam em continuar a trabalhar em um sistema de software após a entrega de seu primeiro lançamento. Portanto, mesmo que a intenção seja manter o time de desenvolvimento junto, as pessoas saem se receberem tarefas de manutenção.

3.4.2 Método ágil e método dirigido por plano

Um requisito fundamental da escalabilidade dos métodos ágeis é integrá-los às abordagens dirigidas por plano. Pequenas empresas iniciantes podem trabalhar com planejamento informal e de curto prazo, mas empresas grandes precisam ter planos e orçamentos de mais longo prazo para investimento, pessoal e evolução comercial. Seu desenvolvimento de software deve apoiar esses objetivos, então o planejamento em longo prazo é essencial.

Os adotantes pioneiros dos métodos ágeis na primeira década do século XXI eram entusiastas e estavam profundamente comprometidos com o manifesto ágil. Eles rejeitavam deliberadamente a abordagem dirigida por plano para a engenharia de software e relutavam em mudar a visão inicial dos métodos ágeis. Entretanto, à medida que as organizações enxergaram o valor e os benefícios de uma abordagem ágil, elas adaptaram esses métodos à sua própria cultura e ao seu modo de trabalhar. Isso foi preciso porque os princípios subjacentes a essa abordagem às vezes são difíceis de concretizar na prática (Figura 3.11).

FIGURA 3.11 Princípios ágeis e a prática organizacional.

Princípio	Prática
Envolvimento do cliente	Isso depende de ter um cliente disposto e capaz de investir tempo com o time de desenvolvimento e que possa representar todos os <i>stakeholders</i> do sistema. Muitas vezes, os representantes dos clientes têm outras demandas e não podem fazer parte do time de desenvolvimento em tempo integral. Nas situações em que existem <i>stakeholders</i> externos, como autoridades reguladoras, é difícil representar suas opiniões para o time ágil.
Acolher as mudanças	Pode ser extremamente difícil priorizar as mudanças, especialmente nos sistemas em que há muitos <i>stakeholders</i> . Geralmente, cada um deles atribui prioridades diferentes a mudanças diferentes.
Entrega incremental	As iterações rápidas e o planejamento de curto prazo do desenvolvimento nem sempre se encaixam nos ciclos de longo prazo do planejamento e marketing empresarial. Os gestores de marketing podem ter de conhecer as características do produto com vários meses de antecedência para preparar uma campanha eficaz.
Manter a simplicidade	Sob a pressão dos cronogramas de entrega, os membros do time podem não ter tempo para realizar simplificações desejáveis no sistema.
Pessoas, não processos	Membros do time podem não ter a personalidade adequada para o envolvimento intenso, o que é característico dos métodos ágeis, e, portanto, podem não interagir bem com os demais membros do time.

Para abordar esses problemas, a maioria dos grandes projetos de desenvolvimento ágil de software combina práticas das abordagens dirigidas por plano e das ágeis. Alguns são basicamente ágeis e outros são praticamente dirigidos por plano, mas com algumas práticas ágeis. Para decidir sobre o equilíbrio entre uma abordagem e outra, é preciso responder a uma série de perguntas técnicas, humanas e organizacionais. Essas questões se relacionam com o sistema que está sendo desenvolvido, com o time de desenvolvimento e com as organizações que estão desenvolvendo e adquirindo esse sistema (Figura 3.12).

FIGURA 3.12 Fatores que influenciam a escolha do desenvolvimento dirigido por plano ou ágil.



Os métodos ágeis foram desenvolvidos e refinados em projetos para desenvolver sistemas de negócio e produtos de software de pequeno e médio porte, em que o desenvolvedor controla a especificação do sistema. Outros tipos de sistema têm atributos como tamanho, complexidade, resposta em tempo real e regulação externa, o que significa que uma abordagem ágil 'pura' provavelmente não funcionará. Será preciso algum planejamento, projeto e documentação antecipados no processo de engenharia de sistemas. Alguns dos pontos-chave são:

1. Qual é o tamanho do sistema que está sendo desenvolvido? Os métodos ágeis são mais eficazes quando o sistema pode ser desenvolvido com um time relativamente pequeno, em uma mesma localização geográfica e que possa se comunicar informalmente. Como isso pode não ser possível nos sistemas grandes, que requerem times de desenvolvimento maiores, talvez seja necessário empregar uma abordagem dirigida por plano.
2. Que tipo de sistema está sendo desenvolvido? Os sistemas que requerem muita análise antes da implementação (por exemplo, sistema de tempo real com requisitos de temporização complexos) normalmente precisam de um projeto bastante detalhado para realizar essa análise. Uma abordagem dirigida por plano pode ser melhor nessas circunstâncias.
3. Qual é a vida útil prevista para o sistema? Os sistemas de vida útil longa podem exigir mais documentação de projeto para comunicar as intenções originais dos desenvolvedores para a equipe de suporte. No entanto, os defensores dos métodos ágeis argumentam que a documentação frequentemente está desatualizada e não tem muita utilidade na manutenção do sistema no longo prazo.
4. O sistema está sujeito a controle externo? Se um sistema tiver de ser aprovado por uma autoridade reguladora externa (por exemplo, a Administração Federal de Aviação norte-americana aprova softwares que são críticos para a operação de uma aeronave), então provavelmente será necessário produzir uma documentação detalhada como parte do plano de segurança (*safety*) do sistema.

Os métodos ágeis colocam uma grande dose de responsabilidade no time de desenvolvimento para que coopere e se comunique durante o desenvolvimento do sistema. Eles contam com as habilidades de engenharia individuais e com o apoio de software para o processo de desenvolvimento. No entanto, na realidade, nem todos são engenheiros altamente qualificados, as pessoas não se comunicam de modo eficaz e nem sempre é possível para os times trabalharem juntos. Pode ser necessário algum planejamento para fazer um uso do pessoal disponível do modo mais eficaz. As questões-chave são:

1. Qual é o nível de competência dos projetistas e programadores do time de desenvolvimento? Às vezes, argumenta-se que os métodos ágeis requerem níveis de habilidade mais altos do que as abordagens dirigidas por plano, nas quais os programadores simplesmente traduzem um projeto detalhado para código. Se houver um time com níveis de habilidade relativamente baixos, pode ser que seja preciso utilizar as melhores pessoas para desenvolver o projeto (*design*), com os outros ficando responsáveis pela programação.
2. Como está organizado o time de desenvolvimento? Se o time estiver distribuído ou se parte do desenvolvimento for terceirizado, então podem ser necessários documentos de projeto para possibilitar a comunicação entre os times.
3. Quais são as tecnologias disponíveis para apoiar o desenvolvimento do sistema? Os métodos ágeis se baseiam, frequentemente, em boas ferramentas para acompanhar um projeto em evolução. Se um sistema estiver sendo desenvolvido usando um IDE que não possua boas ferramentas para visualização e análise do programa, então pode ser necessário gerar mais documentação de projeto.

A televisão e o cinema criaram uma visão popular das empresas de software como organizações informais comandadas por homens jovens (principalmente) que promovem um ambiente de trabalho moderno e elegante, com uma quantidade mínima de burocracia e procedimentos organizacionais. Isso está muito longe da verdade. A maior parte do software é desenvolvida em grandes empresas que estabeleceram suas próprias práticas de trabalho e procedimentos. A gerência nessas empresas pode ficar desconfortável com a falta de documentação e com a tomada de decisão informal que acontece nos métodos ágeis. As questões-chave são:

1. É importante ter uma especificação e um projeto (*design*) bem detalhados antes de passar para a implementação — talvez por motivos contratuais? Se for o caso, provavelmente será preciso usar uma abordagem dirigida por plano para a engenharia de requisitos, mas poderão ser usadas práticas de desenvolvimento ágil durante a implementação do sistema.
2. É realista uma estratégia de entrega incremental, na qual o software é entregue aos clientes ou outros *stakeholders* e um rápido *feedback* é obtido? Os representantes do cliente estarão disponíveis e dispostos a participar do time de desenvolvimento?
3. Existem questões culturais que possam afetar o desenvolvimento do sistema? As organizações de engenharia tradicionais têm uma cultura de desenvolvimento dirigido por plano, já que esta é a norma em engenharia. Normalmente, isso exige ampla documentação do projeto (*design*) em vez de conhecimento informal utilizado nos processos ágeis.

Na realidade, a questão de rotular o projeto como dirigido por plano ou ágil não é muito importante. No final das contas, a preocupação primária dos compradores de um sistema de software é se eles possuem ou não um sistema executável, que

satisfaz suas necessidades, além de fazer coisas úteis para cada usuário ou para a organização. Os desenvolvedores de software devem ser pragmáticos e escolher os métodos mais eficazes para o tipo de sistema a ser desenvolvido, sejam esses métodos rotulados como ágeis ou dirigidos por plano.

3.4.3 Métodos ágeis para sistemas grandes

Os métodos ágeis têm que evoluir para serem utilizados no desenvolvimento de software de larga escala. O motivo fundamental para isso é que esses tipos de software são muito mais complexos e difíceis de compreender e gerenciar do que os sistemas ou produtos de software pequenos. Seis fatores (Figura 3.13) contribuem para essa complexidade:

FIGURA 3.13 Características dos grandes projetos.



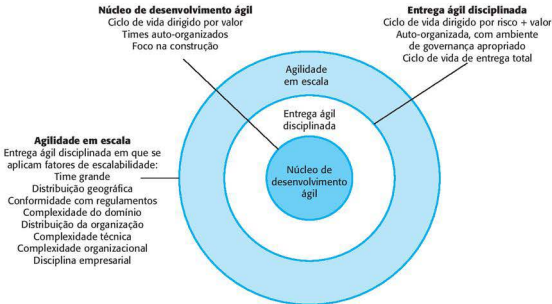
1. Os sistemas grandes normalmente são sistemas de sistemas, ou seja, conjuntos de sistemas separados e que se comunicam, em que diferentes times desenvolvem cada sistema. Frequentemente, esses times trabalham em lugares diferentes, às vezes em fusos horários distintos. É praticamente impossível que cada time tenha uma visão do sistema inteiro. Consequentemente, suas prioridades normalmente são concluir sua parte sem considerar questões sistêmicas mais amplas.
2. Os sistemas grandes são sistemas *brownfield* (HOPKINS; JENKINS, 2008); ou seja, eles incluem e interagem com uma série de sistemas existentes. Muitos dos requisitos do sistema estão relacionados com essa interação e realmente não se prestam à flexibilidade e ao desenvolvimento incremental. Questões políticas também podem ser importantes aqui, já que muitas vezes a solução mais fácil para um problema é mudar um sistema existente. No entanto, isso exige negociação com os gerentes responsáveis para convencê-los de que as mudanças podem ser implementadas sem risco para a operação do sistema.
3. Nas situações em que vários sistemas são integrados para criar um novo, uma fração significativa do desenvolvimento tem a ver com a configuração do sistema em vez do desenvolvimento de código original. Isso não é necessariamente compatível com o desenvolvimento incremental e com a integração frequente do sistema.
4. Os sistemas grandes e seus processos de desenvolvimento frequentemente são restringidos por regras e normas externas que limitam o seu modo de

desenvolvimento, exigem certos tipos de documentação do sistema e assim por diante. Os clientes podem ter requisitos de conformidade específicos que devam ser seguidos e podem exigir documentação do processo.

5. Os sistemas grandes têm um longo tempo de contratação e desenvolvimento. É difícil manter times coerentes que conheçam o sistema ao longo desse período já que, inevitavelmente, as pessoas mudam de emprego e projetos.
6. Esses sistemas normalmente têm um conjunto diverso de *stakeholders*, com diferentes perspectivas e objetivos. Por exemplo, profissionais de enfermagem e administradores podem ser os usuários finais de um sistema clínico, mas os médicos mais antigos, gestores hospitalares e outros também são *stakeholders* no sistema. É praticamente impossível envolver todos eles no processo de desenvolvimento.

Dean Leffingwell, que tem uma grande dose de experiência em escalabilidade de métodos ágeis, desenvolveu o *Scaled Agile Framework* (LEFFINGWELL, 2007, 2011) para apoiar o desenvolvimento de software em grande escala e múltiplos times. Ele relata como esse método tem sido utilizado com êxito em uma série de empresas grandes. A IBM também desenvolveu um arcabouço para uso em larga escala dos métodos ágeis, chamado *Agile Scaling Model* (ASM). A Figura 3.14, extraída do artigo de Amber que discute o ASM (AMBER, 2010), mostra uma visão geral desse modelo.

FIGURA 3.14 Modelo de agilidade em escala da IBM (© IBM 2010).



O ASM reconhece que a escalabilidade é um processo que ocorre em etapas, em que os times de desenvolvimento passam das práticas ágeis fundamentais discutidas aqui ('núcleo de desenvolvimento ágil'), para a chamada 'entrega ágil disciplinada'. Essencialmente, essa etapa envolve a adaptação dessas práticas a um contexto organizacional disciplinado e o reconhecimento de que os times não podem simplesmente focar no desenvolvimento, mas também devem levar em conta outras etapas do processo de engenharia de software, como os requisitos e o projeto de arquitetura.

A etapa final da escalabilidade no ASM é a passagem para a 'agilidade em escala', quando é reconhecida a complexidade inerente aos projetos grandes. Isso envolve levar em consideração fatores como o desenvolvimento distribuído, ambientes legados complexos e requisitos de conformidade com regulamentos. As práticas utilizadas na entrega ágil disciplinada podem precisar ser modificadas de acordo com o projeto para levar esses fatores em conta e, às vezes, outras práticas dirigidas por plano precisam ser adicionadas ao processo.

Nenhum modelo individual é adequado para todos os produtos ágeis de larga escala, pois o tipo de produto, os requisitos do cliente e as pessoas disponíveis são todos diferentes. No entanto, as abordagens para escalabilidade dos métodos ágeis têm uma série de coisas em comum:

1. Uma abordagem completamente incremental para a engenharia de requisitos é impossível. É essencial que haja algum trabalho inicial sobre requisitos de software. Esse trabalho é necessário para identificar as diferentes partes do sistema que podem ser desenvolvidas por diferentes times e, frequentemente, para fazer parte do contrato de desenvolvimento do sistema. Contudo, esses requisitos normalmente não devem ser especificados em detalhes, mas em incrementos.
2. Não pode haver um único *Product Owner* ou representante do cliente. Pessoas diferentes precisam se envolver nas diferentes partes do sistema e se comunicar continuamente e negociar durante todo o processo de desenvolvimento.
3. Não é possível se concentrar apenas no código do sistema. É preciso que haja mais projeto (*design*) antecipado e documentação do sistema. A arquitetura do software precisa ser projetada e deve haver documentação produzida para descrever os aspectos críticos do sistema, como os esquemas de bancos de dados e a distribuição do trabalho entre os times.
4. Os mecanismos de comunicação entre os times precisam ser projetados e utilizados. Isso deve envolver conferências regulares por telefone e vídeo entre os membros do time e reuniões *on-line* curtas e frequentes, nas quais os times atualizam uns aos outros sobre seus progressos. Uma gama de canais de comunicação, como e-mail, mensagens instantâneas, *wikis* e redes sociais deve ser disponibilizada para facilitar as comunicações.
5. Integração contínua, em que o sistema inteiro é construído toda vez que qualquer desenvolvedor confere uma mudança, é praticamente impossível quando vários programas diferentes precisam ser integrados para criar o sistema. No entanto, é essencial construir frequentemente o sistema e fazer lançamentos regulares desse sistema. As ferramentas de gerenciamento de configuração que apoiam o desenvolvimento de software por múltiplos times são essenciais.

O Scrum tem sido adaptado ao desenvolvimento em larga escala. Em essência, o modelo de time Scrum, descrito na Seção 3.3, é mantido, mas vários times Scrum são estabelecidos. As características fundamentais do Scrum multimes são as listadas a seguir.

1. *Replicação de papéis.* Cada time tem um *Product Owner* para o seu componente de trabalho e um *Scrum Master*. Pode haver um *Product Owner* principal e um *Scrum Master* para o projeto inteiro.

2. *Arquitetos de produto.* Cada time opta por um arquiteto de produto e eles colaboram para projetar e desenvolver a arquitetura global do sistema.
3. *Alinhamento de entregas.* As datas das entregas do produto de cada time são alinhadas para que seja produzido um sistema demonstrável e completo.
4. *Scrum de Scrums.* Há um Scrum dos Scrums diário, no qual representantes de cada time se reúnem para discutir o progresso, identificar problemas e planejar o trabalho a ser feito naquele dia. Os Scrums de cada time podem ser escalonados em horários diferentes para que os representantes de outros times possam estar presentes, se necessário.

3.4.4 Métodos ágeis nas organizações

As pequenas empresas que desenvolvem produtos de software estão entre os adeptos mais entusiasmados dos métodos ágeis. Essas empresas não limitadas pelas burocracias organizacionais ou pelos padrões de processo podem mudar rapidamente para adotar novas ideias. Naturalmente, empresas maiores também experimentaram métodos ágeis em projetos específicos, mas é mais difícil para elas introduzi-los na organização.

Essa dificuldade acontece por uma série de razões:

1. Os gerentes de projeto, que não têm experiência com métodos ágeis, podem relutar em aceitar o risco de uma nova abordagem, já que não sabem como isso vai afetar seus projetos particulares.
2. As grandes organizações frequentemente têm procedimentos e padrões de qualidade que todos os projetos devem seguir e, devido à sua natureza burocrática, esses procedimentos e padrões tendem a ser incompatíveis com os métodos ágeis. Às vezes eles são apoiados por ferramentas de software (por exemplo, ferramentas de gerenciamento dos requisitos) e o uso delas é obrigatório em todos os projetos.
3. Os métodos ágeis parecem funcionar melhor quando os membros do time têm um nível de habilidade relativamente elevado. No entanto, dentro das grandes organizações, é comum haver uma ampla gama de habilidades e capacidades, e as pessoas com níveis de habilidade mais baixos podem não ser membros de time eficazes nos processos ágeis.
4. Pode haver uma resistência cultural aos métodos ágeis, especialmente nas organizações que tenham um longo histórico de uso de processos convencionais de engenharia de sistemas.

O gerenciamento de mudança e os procedimentos de teste são exemplos de procedimentos da empresa que podem não ser compatíveis com os métodos ágeis. O gerenciamento de mudança é o processo de controlar as mudanças de um sistema, tal que o seu impacto seja previsível e os custos, controlados. Todas as mudanças devem ser aprovadas antes de serem implantadas e isso entra em conflito com a noção de refatoração. Quando ela faz parte de um processo ágil, qualquer desenvolvedor pode melhorar qualquer código sem obter aprovação externa. Nos sistemas grandes também há um padrão de testes no qual um *build* do sistema (um executável completo) é entregue a uma equipe de testes externa. Isso pode entrar

em conflito com as abordagens de teste *a priori* (*test-first*) utilizadas nos métodos de desenvolvimento ágil.

Introduzir e manter o uso dos métodos ágeis em uma grande organização é um processo de mudança de cultura. Leva muito tempo para que esta seja implementada e isso frequentemente requer uma troca na gerência antes de ser feito. As empresas que desejam usar métodos ágeis precisam de 'evangelizadores' para promover a mudança. Em vez de tentar impor esses métodos em desenvolvedores relutantes, as empresas constataram que a melhor maneira de introduzi-los é aos poucos, começando com um grupo de desenvolvedores entusiastas dessas abordagens. Um projeto ágil bem-sucedido pode ser como um ponto de partida, com o time de projeto espalhando a prática ágil pela organização. Depois que o conceito de desenvolvimento ágil for conhecido, de um modo geral, ações explícitas podem ser adotadas para disseminá-lo na organização.

PONTOS-CHAVE

- ▶ Os métodos ágeis são de desenvolvimento iterativo e se concentram na redução de sobrecarga no processo e na documentação, bem como na entrega incremental de software. Eles envolvem representantes do cliente diretamente no processo de desenvolvimento.
- ▶ A decisão de usar uma abordagem ágil ou uma dirigida por plano para o desenvolvimento deve depender do tipo de software que está sendo desenvolvido, da capacidade do time de desenvolvimento e da cultura da empresa que está desenvolvendo o sistema. Na prática, pode-se utilizar uma mistura de técnicas ágeis e dirigidas por plano.
- ▶ As práticas de desenvolvimento ágil incluem requisitos expressados na forma de histórias do usuário, programação em pares, refatoração, integração contínua e desenvolvimento com testes *a priori* (*test-first*).
- ▶ O Scrum é um método ágil que define um arcabouço para organizar projetos ágeis. Ele gira em torno de *sprints*, que são períodos de tempo fixos nos quais um incremento do sistema é desenvolvido. O planejamento se baseia em priorizar um *backlog* de trabalho e selecionar as tarefas de maior prioridade para uma *sprint*.
- ▶ Para promover a escalabilidade de métodos ágeis, algumas práticas dirigidas por plano precisam ser integradas à prática ágil. Elas incluem requisitos antecipados, vários representantes do cliente, mais documentação, ferramental comum aos times de projeto e o alinhamento das entregas entre os times.

LEITURAS ADICIONAIS

"Get ready for agile methods, with care." Uma crítica ponderada dos métodos ágeis que discute seus pontos fortes e fracos, escrita por um engenheiro de software com vasta experiência. Ainda muito relevante, embora com quase 15 anos de idade. BOEHM, B. *IEEE Computer*, jan. 2002. doi:10.1109/2.976920.

Extreme Programming explained. Esse foi o primeiro livro sobre Programação Extrema e talvez ainda seja o de leitura mais agradável. Ele explica a abordagem pela perspectiva de um de seus criadores e seu entusiasmo brota claramente do livro. BECK, K.; ANDRES, C. Addison-Wesley, 2004.

Essential Scrum: a practical guide to the most popular agile process. Trata-se de uma descrição abrangente e didática do progresso, em 2011, do método Scrum. RUBIN, K. S. Addison-Wesley, 2013.

"Agility at scale: economic governance, measured improvement and disciplined delivery." Esse artigo discute a abordagem da IBM para escalabilidade dos métodos ágeis; eles têm uma abordagem sistemática para integrar o desenvolvimento ágil com o dirigido por plano. É uma discussão excelente e ponderada sobre as questões-chave na escalabilidade do desenvolvimento ágil. BROWN, A.W.; AMBLER, S.W.; ROYCE, W. *Proc. 35th Int. Conf. on Software Engineering*, 2013. doi:10.1145/12944.12948.

SITE²

Apresentações em PowerPoint para este capítulo disponíveis em: <<http://software-engineering-book.com/slides/chap3/>>.

Links para vídeos de apoio disponíveis em: <<http://software-engineering-book.com/videos/agile-methods/>>.

² Todo o conteúdo disponibilizado na seção *Site* de todos os capítulos está em inglês.

EXERCÍCIOS

- 3.1 Explique por que a entrega e implantação rápidas dos novos sistemas costuma ser mais importante para as empresas do que a sua funcionalidade detalhada.
- 3.2 Explique como os princípios relacionados aos métodos ágeis levam ao desenvolvimento e implantação acelerados do software.
- 3.3 A Programação Extrema apresenta os requisitos do usuário como histórias, com cada uma delas escrita em um cartão de história. Discuta as vantagens e desvantagens dessa abordagem para a descrição dos requisitos.
- 3.4 Explique por que o desenvolvimento com testes *a priori* (*test-first*) ajuda o programador a desenvolver uma melhor compreensão dos requisitos do sistema. Quais são as possíveis dificuldades com o desenvolvimento com testes desse tipo?
- 3.5 Sugira quatro motivos para a produtividade dos programadores que trabalham em pares poder ser mais da metade da produtividade de dois programadores trabalhando individualmente.
- 3.6 Compare a abordagem Scrum para o gerenciamento de projetos com as abordagens convencionais dirigidas por plano, conforme discutido no Capítulo 23. Sua comparação deve se basear na eficácia de cada abordagem para planejar a alocação das pessoas aos projetos, estimar os custos, manter a coesão do time e gerenciar mudanças nos times.
- 3.7 Para reduzir custos e o impacto ambiental do deslocamento para o trabalho, sua empresa decide fechar vários escritórios e autorizar o pessoal de suporte a trabalhar em casa. No entanto, a alta gestão que introduz as políticas não sabe que o software é desenvolvido usando Scrum. Explique como você poderia usar a tecnologia para apoiar o Scrum em um ambiente distribuído para tornar isso possível. Quais problemas você provavelmente encontrará ao usar essa abordagem?
- 3.8 Por que é necessário introduzir alguns métodos e documentação das abordagens dirigidas por plano quando escalamos métodos ágeis para projetos maiores desenvolvidos por times distribuídos?
- 3.9 Explique por que pode ser difícil usar métodos ágeis em um projeto grande, para desenvolver um novo sistema de informações que faz parte de um sistema de sistemas organizacionais.
- 3.10 Um dos problemas de ter um usuário estreitamente envolvido com um time de desenvolvimento de software é que eles 'se tornam nativos', ou seja, adotam a perspectiva do time de desenvolvimento e perdem de vista as necessidades dos seus colegas usuários. Sugira três maneiras de evitar este problema e discuta as vantagens e desvantagens de cada abordagem.

REFERÊNCIAS

- AMBLER, S. W. "Scaling agile: a executive guide." 2010. Disponível em: <https://www.ibm.com/developerworks/community/blogs/ambler/entry/scaling_agile_an_executive_guide10/>. Acesso em: 30 mar. 2018.
- ARISHOLM, E.; GALLIS, H.; DYBA, T.; SJÖBERG, D. I. K. "Evaluating pair programming with respect to system complexity and programmer expertise." *IEEE Trans. on Software Eng.*, v. 33, n. 2, 2007. p. 65-86. doi:10.1109/TSE.2007.17.
- BECK, K. "Chrysler goes to 'extremes.'" *Distributed computing*, v. 10, 1998, p. 24-28.
- _____. "Embracing change with extreme programming." *IEEE Computer*, v. 32, n. 10, 1999. p. 70-78. doi:10.1109/2.796139.
- BELLOUITI, S. "How Scrum helped our A-team." 2009. Disponível em: <<http://www.scrumalliance.org/community/articles/2009/2009-june/how-scrum-helped-our-team/>>. Acesso em: 30 mar. 2018.
- BIRD, J. "You can't be agile in maintenance." 2009. Disponível em: <<http://swreflections.blogspot.co.uk/2011/10/you-cant-be-agile-in-maintenance.html>>. Acesso em: 30 mar. 2018.
- DEEMER, P. "The distributed Scrum primer." 2011. Disponível em: <<http://www.goodagile.com/distributedscrumprimer/>>. Acesso em: 30 mar. 2018.
- FOWLER, M.; BECK, K.; BRANT, J.; OPDYKE, W.; ROBERTS, D. *Refactoring: improving the design of existing code*. Boston: Addison-Wesley, 1999.

- HOPKINS, R.; JENKINS, K. *Eating the IT elephant: moving from Greenfield development to Brownfield*. Boston: IBM Press, 2008.
- JEFFRIES, R.; MELNIK, G. "TDD: The art of fearless programming." *IEEE Software*, v. 24, 2007. p. 24-30. doi:10.1109/MS.2007.75.
- KILNER, S. "Can agile methods work for software maintenance." 2012. Disponível em: <<http://www.vlegaci.com/can-agile-methods-work-for-software-maintenance-part-1/>>. Acesso em: 17 fev. 2015.
- LARMAN, C.; BASILI, V. R. "Iterative and incremental development: a brief history." *IEEE Computer*, v. 36, n. 6, 2003. p. 47-56. doi:10.1109/MC.2003.1204375.
- LEFFINGWELL, D. *Scaling software agility: best practices for large enterprises*. Boston: Addison-Wesley, 2007.
- _____. *Agile software requirements: lean requirements practices for teams, programs and the enterprise*. Boston: Addison-Wesley, 2011.
- MULDER, M.; VAN VLIET, M. "Case Study: distributed Scrum project for dutch railways." *InfoQ*. 2008. Disponível em: <<http://www.infoq.com/articles/dutch-railway-scrum>>. Acesso em: 30 mar. 2018.
- RUBIN, K. S. *Essential Scrum*. Boston: Addison-Wesley, 2013.
- SCHATZ, B.; ABDELSHAFI, I. "Primavera gets agile: a successful transition to agile development." *IEEE Software*, v. 22, n. 3, 2005. p. 36-42. doi:10.1109/MS.2005.74.
- SCHWABER, K.; BEEDLE, M. *Agile software development with Scrum*. Englewood Cliffs, NJ: Prentice-Hall, 2001.
- STAPLETON, J. *DSDM: business focused development*. 2. ed. Harlow, UK: Pearson Education, 2003.
- TAHCHEV, P.; LEME, F.; MASSOL, V.; GREGORY, G. *JUnit in action*. 2. ed. Greenwich, CT: Manning Publications, 2010.
- WEINBERG, G. *The psychology of computer programming*. New York: Van Nostrand, 1971.
- WILLIAMS, L.; KESSLER, R. R.; CUNNINGHAM, W.; JEFFRIES, R. "Strengthening the case for pair programming." *IEEE Software*, v. 17, n. 4, 2000. p. 19-25. doi:10.1109/52.854064.